

TCP 和 UDP 的区别

1. TCP 和 UDP 的概念：

- **TCP**（传输控制协议）：**面向连接的可靠**传输协议，保证数据完整有序到达
通过三次握手建立连接，提供端到端的可靠传输，确保数据无差错、不丢失、按序到达。
- **UDP**（用户数据报协议）：**无连接的不可靠**传输协议，注重传输速度和简单性
直接发送数据包，不建立连接，不保证传输质量，但延迟极低，适用于实时性要求高的场景

2. 区别：

- 连接方式：
 - TCP：需要先通过三次握手建立连接（SYN、SYN-ACK、ACK），数据传输完成后，通过四次挥手释放连接(FIN、ACK、FIN-ACK、ACK)。
 - UDP：直接发送数据包，不需要握手挥手
- 可靠性：
 - TCP：通过**确认应答、超时重传、数据校验**确保数据可靠
 - UDP：无重传机制，发送即丢弃、可靠性由应用层处理
- 传输顺序：
 - TCP：通过**序列号**确保接收端按序重组数据
 - UDP：不维护数据顺序，接收端可能乱序接收
- 头部开销：
 - TCP：头部最小20字节，包含序列号、确认号、窗口大小等字段
 - UDP：头部固定8字节（源/目标端口、长度、校验和）
- 流量控制：
 - TCP：通过**滑动窗口**动态调整发送速率，避免接收方缓冲区溢出
 - UDP：无流量控制，可能因接收方处理不及时导致丢包
- 拥塞控制：
 - TCP：通过慢启动、拥塞避免、快重传等算法避免网络拥堵
 - UDP：无拥塞控制，可能加剧拥塞
- 传输速度：
 - TCP：由于连接管理、流量控制，拥塞控制、重传等机制，传输延迟较高
 - UCP：无控制开销，传输速度极快
- 应用场景：
 - TCP：HTTP/HTTPS、SMTP（邮件）、FTP（文件传输）
 - UCP：DNS查询、视频会议、在线游戏、直播

HTTP 请求方式

HTTP/1.1 最初定义的8种核心请求方法

1. **GET**: 获取资源, 参数在URL中, 安全 (不修改服务器资源) 且幂等 (多次请求结果一致)
 - 作用: 请求指定资源, 参数通过URL传递
 - 特点: 安全 (不修改服务器资源)、幂等 (多次请求结果一致)、可缓存 (响应可被浏览器或代理缓存)
 - 场景: 数据查询、页面加载
2. **POST**: 提交数据 (如表单数据), 可能修改服务器状态
 - 作用: 提交数据 (请求体携带数据)
 - 特点: 不安全 (可能修改服务器状态, eg: 新增数据)、不幂等 (重复提交可能产生不同结果, eg: 多次新增数据)
 - 场景: 表单提交、触发业务逻辑
3. **PUT**: 全量更新资源, 幂等
 - 作用: 全量更新资源 (客户端需提供完整数据)
 - 特点: 幂等、不安全
 - 场景: 用户信息全量更新
4. **DELETE**: 删除资源, 幂等
 - 作用: 删除指定资源
 - 特点: 幂等、不安全
 - 场景: 删除文章、用户等资源
5. **HEAD**: 类似GET, 只返回响应头
 - 作用: 与GET类似, 但服务器不返回响应体, 仅返回响应头
 - 特点: 轻量、安全、幂等
 - 场景: 检查资源是否存在或更新, eg: 验证链接有效性、资源缓存状态
6. **OPTIONS**: 查询服务器支持的请求方法
 - 作用: 查询服务器支持的请求方法, 或检查跨域请求权限
 - 特点: 预检请求: 浏览器自动发送OPTIONS请求进行CORS验证
 - 场景: 跨域API调用前的预检请求
7. **TRACE**: 回显请求
 - 作用: 回显客户端请求 (用于调试)
 - 特点: 易引发安全风险 (如XST攻击), 通常禁用。
8. **CONNECT**: 建立代理通道
 - 作用: 建立代理隧道 (如HTTPS)
 - 特点: 由服务器/代理处理, 不直接暴露给业务层

GET和POST区别

1. GET 和 POST 的概念:

- GET请求:
 - 作用: 请求指定资源, 参数通过URL传递
 - 特点:

- 参数通过URL传递，格式：key=value&key=value
- 不安全（传输数据的信息直接暴露在了浏览器的地址栏）
- 数据长度受限制（不同浏览器规定不同，一般2k个字符）
- 场景：数据查询，页面加载
- POST请求：
 - 作用：提交数据（请求体携带数据）
 - 特点：
 - 提交的数据是在 http 请求中，不会展示在地址栏。数据真正存放在请求头后面的请求体中。请求体一般在载荷中。
 - 相对于 GET 请求更安全（数据不会直接显示在浏览器的地址栏）
 - 理论上没有数据长度的限制
 - 场景：表单提交、触发业务逻辑

2. 区别：

1. 用途和语义：

- GET请求主要用于从服务器获取资源，如网页内容、图片或API数据。它应该是**安全**和**幂等**的，意味着多次相同的GET请求应该返回相同的结果，且不会改变服务器状态
- POST主要用于向服务器提交数据，如提交表单或上传文件。它可能会改变服务器状态，因此不是幂等的

2. 数据传输方式：

- GET请求将参数附加在URL中
- POST请求将数据封装在HTTP请求的请求体中，不会出现在URL中

3. 数据大小限制：

- GET请求受URL长度限制，具体限制因浏览器和服务器而异，通常在2048-8092字符之间
- POST请求理论上没有数据大小限制，实际限制取决于服务器配置

4. 安全性：

- 从**数据暴露的安全性**考虑，GET参数在URL中可见，不适合传输敏感信息，如密码。从**服务器状态修改的安全性**考虑，GET请求是幂等的，不修改服务器资源，因此安全
- 从**数据暴露的安全性**考虑，POST请求的数据不会显示在URL中，相对更安全，但两者都不是绝对安全的，敏感数据应该使用HTTPS传输。从**服务器状态修改的安全性**考虑，POST请求可修改服务器状态，不安全

5. 缓存：

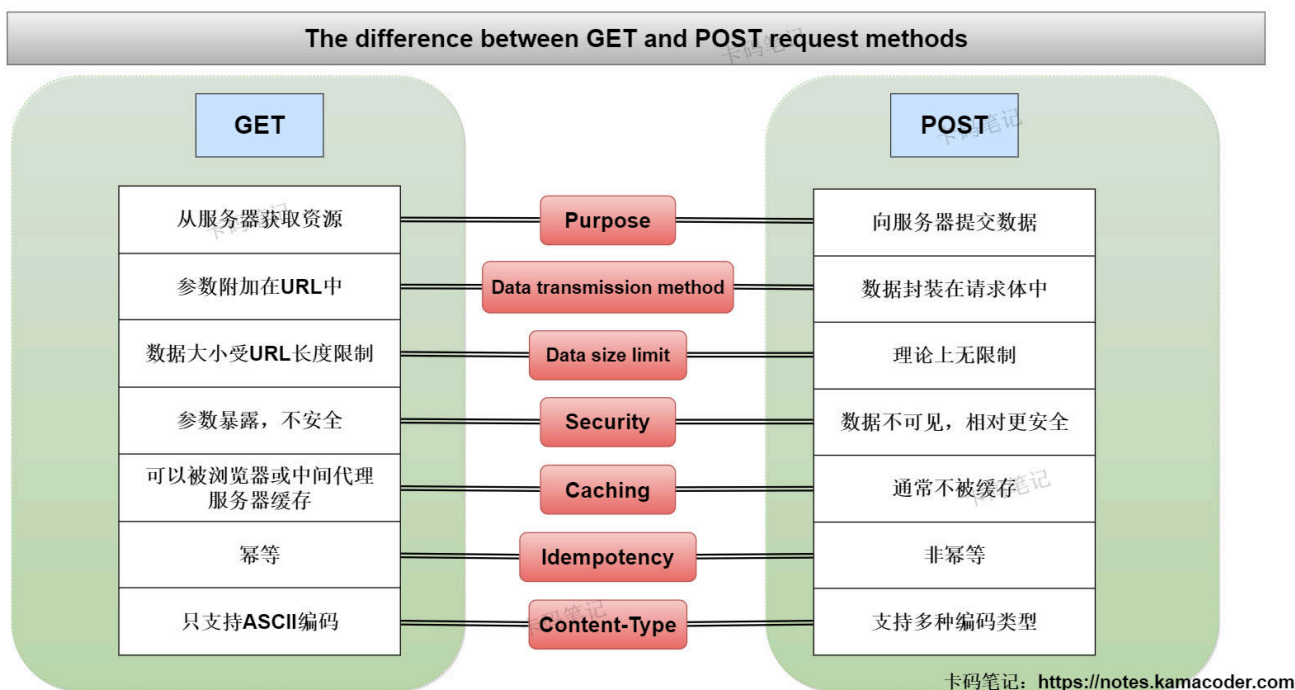
- GET请求可以被**浏览器**缓存，也可以被**中间代理服务器**缓存，有利于提高效率，缓存**服务器返回的数据**（通常是响应体）
- POST请求通常不缓存，每次都会向服务器发送新的请求

6. 幂等性：

- GET是**幂等**的，多次相同的GET请求应该返回相同的结果，不会改变服务器为状态
- POST**不是幂等**的，多次相同的POST请求可能会导致多次修改服务器状态或多次提交数据

7. 编码类型：

- GET只允许ASCII字符，对于非ASCII字符需要进行URL编码
- POST支持多种编码类型，可以发送复杂数据结构，如JSON或XML



HTTP状态码

1. 2xx（成功类）

1. **200：OK**，表示请求已被正确处理（如网页加载成功、API成功返回数据）

2. 3xx（重定向类）

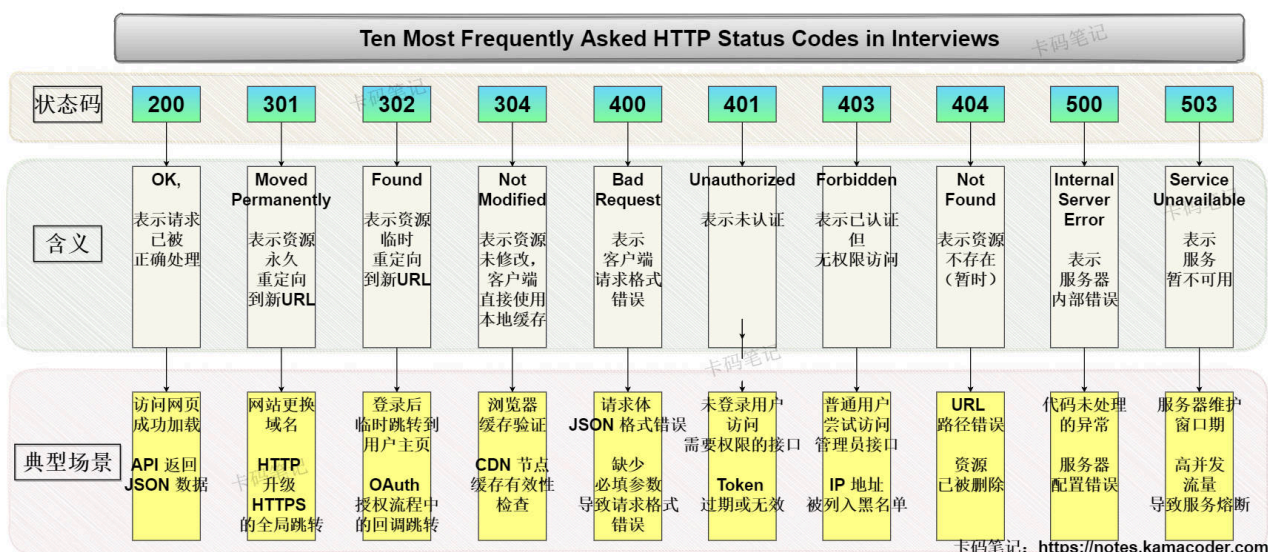
1. **301：Moved Permanently**，表示资源永久重定向到新的URL（如网站更换域名、旧链接跳转）
2. **302：Found**，表示资源临时重定向到新的URL，客户端后续请求可能仍访问原URL（登录后跳转回原页面）
3. **304：Not Modified**，表示资源未修改，客户端可直接使用缓存副本（浏览器缓存验证，缓存机制的核心状态码）

3. 4xx（客户端错误类）

1. **400：Bad Request**，表示客户端请求语法或参数错误（缺少必填参数、请求体JSON格式错误）
2. **401：Unauthorized**，请求未提供有效身份凭证，需认证后重试（未登录用户访问需要权限的接口、Token过期或无效）
3. **403：Forbidden**，客户端身份已认证，但无权访问目标资源（普通用户尝试访问管理员接口、IP地址被列入黑名单）
4. **404：Not Found**，服务器未找到请求的资源（URL路径错误、资源已被删除）

4. 5xx（服务器错误类）

1. **500: Internal Server Error**，服务器内部错误，无法完成请求（代码未处理的异常、服务器配置错误）
2. **503: Service Unavailable**，服务器暂时无法处理请求（通常因过载或维护）



TCP三次握手流程

1. TCP三次握手:

首先服务器要进入**LISTEN**状态，等待客户端的连接

1. **第一次握手 (SYN)**: 客户端向服务器发送连接请求报文段，其中同步位 $SYN = 1$ ，初始序列号 $seq = x$ ，SYN报文发送后，客户端进入**SYN-SENT**状态
2. **第二次握手 (SYN-ACK)**: 服务器收到客户端的连接请求后，如果同意建立连接，则向客户端发送一个**确认报文段**，其中同步位 $SYN = 1$ ，**确认位** $ACK = 1$ 。由于TCP是全双工通信，服务器也可以向客户端发送数据，所以服务器设置一个初始序列号 $seq = y$ ，确认号 $ack = x + 1$ （表示来自客户端的编号在 x 之前的所有字节都已经正确接收，现在服务器希望得到编号为 $x + 1$ 的字节）。SYN-ACK报文发送后，服务器进入**SYN-RCVD**状态
3. **第三次握手 (ACK)**: 客户端收到服务器发来的确认报文段后，需要再向服务器发送**确认报文段**，其中**确认位** $ACK = 1$ ，序号 $seq = x + 1$ ，确认号 $ack = y + 1$ 。客户端发送完ACK报文后，进入**ESTABLISHED**状态；当服务器收到该报文后，也进入**ESTABLISHED**状态。此时TCP连接建立成功，双方可以进行数据传输

2. 为什么不是两次握手

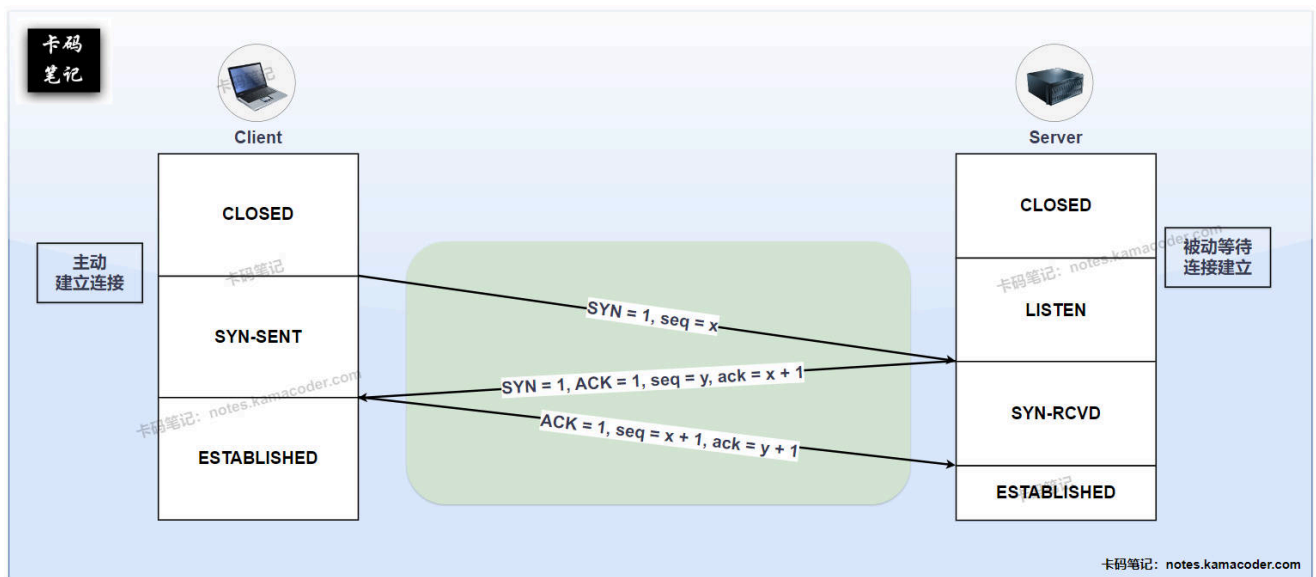
- 两次握手无法确认客户端的接收能力
 - 三次握手的核心目的是**让双方都确认自己和对方的发送/接收能力正常**
 - **第一次握手**: 客户端发SYN。服务器知道: 客户端**能发**（不知道客户端能不能收）
 - **第二次握手**: 服务器回SYN-ACK。客户端收到后，知道: 服务器**能收能发**
 - **第三次握手**: 客户端回ACK。服务端收到后，才知道: 客户端**能收**
- 如果只有两次握手（即服务端发完 $SYN+ACK$ 就算连接建立），**服务端永远无法确认客户端是否能正常接收消息**
- 无法避免网络中滞留的**旧连接请求**突然到达而导致服务端**长时间等待**

- 场景模拟：

1. **第一次连接请求（慢）**：客户端发送一个 SYN（序号=100）请求连接。这个包在网络中卡住了（网络拥塞）。
2. **客户端超时重传**：客户端等了一会儿没收到回应，于是重新发送一个新的 SYN（序号=200）。
3. **新连接成功建立并关闭**：服务器收到序号=200 的请求，建立连接，传输数据，然后正常关闭。此时连接已结束。
4. **旧请求突然到达（致命）**：那个被卡住的旧 SYN（序号=100）此时才到达服务端。

- 如果采用两次握手：

- 服务端收到这个过期的 SYN（序号=100），以为客户端又想建一个新连接。由于只有两次握手，服务端就直接回复 SYN+ACK（确认序号=101），并且认为连接已经建立（进入 ESTABLISHED 状态）。
- 服务端会开始等待客户端发送数据，或主动发送数据。但客户端根本不认为这个连接存在（客户端早就不记得序号100的事了），收到服务端的 SYN+ACK 会直接丢弃或回复 RST 拒绝。
- 结果：服务端白白浪费了资源（内存、端口），等待一个永远不会来的数据



TCP四次挥手流程

1. TCP四次挥手：（客户端为主动关闭方）

1. 第一次挥手（Client --> Server: FIN）：

- 发送一个**连接释放报文段**，终止位 $FIN = 1$ ，初始序号 $seq = u$
- 客户端发送 FIN 报文后，进入 **FIN-WAIT-1** 状态，等待服务器确认
- 此时客户端不能再发送数据，但是可以接收数据

2. 第二次挥手（Server --> Client: ACK）：

- 服务器收到客户端的 FIN 报文后，返回一个**确认报文段**，确认位 $ACK = 1$ ，序列号 $seq = v$ ，确认号 $ack = u + 1$
- 服务器发送 ACK 报文后，进入 **CLOSE-WAIT** 状态

- 客户端收到 ACK 报文后，进入**FIN-WAIT-2**状态，等待服务器的连接释放报文段 FIN-ACK

3. 第三次挥手 (Server --> Client: FIN-ACK):

- 发送一个连接释放报文段，终止位 $FIN = 1$ ，确认位 $ACK = 1$ ， $seq = w$ ， $ack = u + 1$
- 服务器发送 FIN 报文后，进入**LAST-ACK**状态，等的客户端确认

4. 第四次挥手 (Client --> Server: ACK):

- 客户端收到服务器的 FIN-ACK 报文后，必须再向服务器发送一个确认报文段，确认位 $ACK = 1$ ，序列号 $seq = u + 1$ ，确认号 $ack = w + 1$
- 客户端发送完确认报文段后，进入 **TIME-WAIT** 状态
- 服务器收到确认报文段后，连接关闭，进入 **CLOSED** 状态
- 客户端在 **TIME-WAIT** 状态下等待 2 MSL (Maximum Segment Lifetime, 报文最大生存时间) 后，才最终进入 **CLOSED** 状态。
- **TIME-WAIT** 状态的存在是为了**确保服务器能收到最后的 ACK 报文**，如果该报文丢失，服务器会超时重传它的 FIN-ACK 报文 (第三次挥手)。客户端在 **TIME-WAIT** 状态可以响应这个重传的 FIN-ACK，再次发送 ACK，确保服务器能正常关闭。

2. 四次挥手必要性:

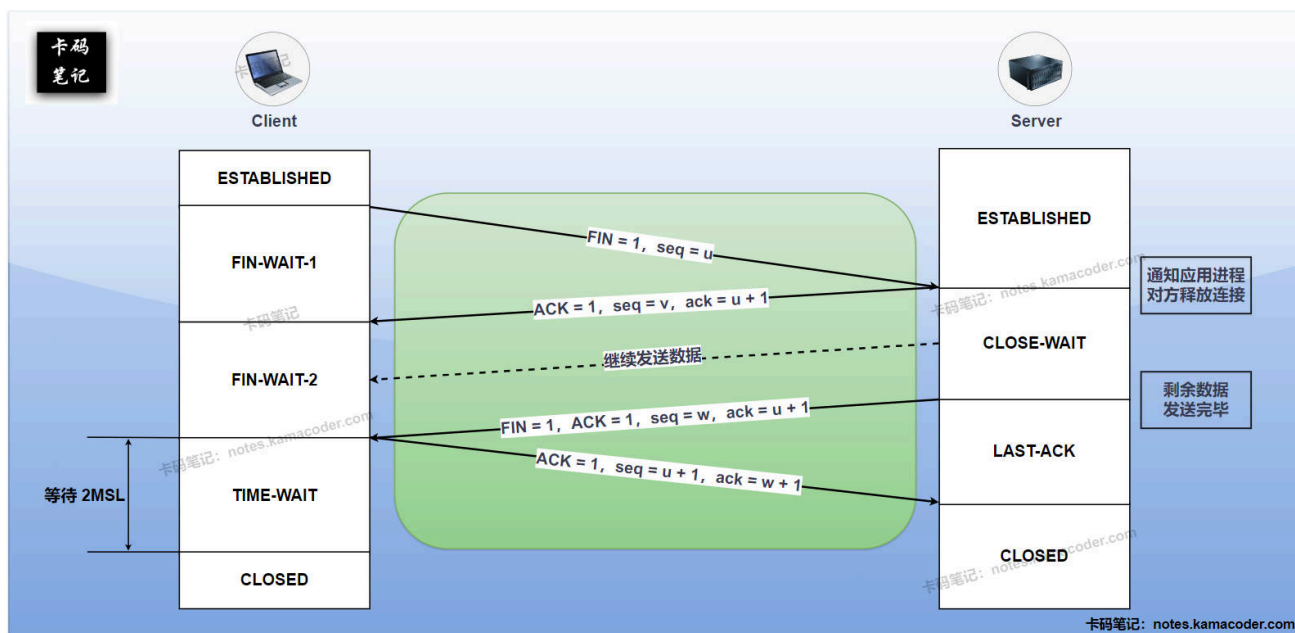
- 因为 TCP 是**全双工**的，每一端的发送通道需要独立关闭 —— 两个 FIN 和两个 ACK，刚好四次

3. 为什么不是三次:

- 客户端发 FIN 后，服务器可能仍有数据要发，需先回复 ACK，处理完数据后，再发连接释放报文段 FIN。
- 若合并第二次和第三次挥手，可能迫使服务器立即关闭，导致数据丢失。

4. 为什么不是五次:

- 客户端和服务器各发一次 FIN 和 ACK，能明确关闭双向连接。五次挥手会增加冗余步骤 (例如重复确认)，但不会提升可靠性



TCP连接如何确保可靠性

TCP拥塞控制实现

1. 拥塞控制定义与目的:

- 定义:
 - TCP拥塞控制是TCP协议中为了防止发送方向网络中注入过多数据，导致网络过载而性能下降的一系列机制和算法
 - 通过**动态调整发送速率**（拥塞窗口）来实现
- 目的:
 - **防止拥塞崩溃**：根本目的是防止过多的数据同时进入网络，超出网络的处理能力而丢包
 - **提高网络效率**：在不导致拥塞的情况下，尽可能充分的利用网络可用带宽，实现较高的传输效率
 - **保证相对公平性**：理想情况下，**多个** 竞争同一瓶颈链路带宽的 **TCP 连接** 应该能够相对公平地共享带宽资源。拥塞控制算法的设计也需要考虑这一点

2. TCP拥塞控制四种经典算法:

1. 慢启动:

- 启动条件：连接刚刚建立，或检测到超时丢包后
- 机制:
 - 初始**拥塞窗口cwnd**为一个较小的值，先发送少量数据，如果没有发生拥塞，则以**指数方式**快速增长发送速率（拥塞窗口 cwnd），目的是快速探测网络的可用带宽
 - 当 cwnd 增长到**等于或超过**慢开始阈值 ssthresh 时，或者当检测到丢包时，慢开始阶段结束。如果是因为达到 ssthresh，则进入拥塞避免阶段

2. 拥塞避免:

- 启动条件：当 `cwnd` 达到 `ssthresh` 后，从慢开始算法切换过来
- 机制：
 - 采用**线性**方式增长拥塞窗口
 - 目的是在接近网络容量时更谨慎地探测，避免轻易导致拥塞

3. 快重传：

- 启动条件：发送方连续收到了**三个或更多的重复 ACK**，表示接收方收到了后续的数据段，没有收到当前ACK对应的数据
- 机制：一旦收到 3 个重复 ACK，发送方就**不等重传计时器超时**，立即**重传**那个被认为丢失的数据段

4. 快恢复：

- 启动条件：在执行了快重传之后
- 机制：
 - 快恢复认为收到 3 个重复ACK表明网络可能只是发生了少量丢包，而不是彻底拥塞
 - 此时会将**慢开始阈值 `ssthresh` 设置为当前 `cwnd` 的一半**，将**拥塞窗口 `cwnd` 也减少到当前 `cwnd` 的一半**
 - 进入拥塞避免阶段

HTTPS和HTTP的区别

1. 概念：

1. **HTTP**：超文本传输协议，用于客户端（如浏览器）与服务器之间的数据传输，**明文传输**
2. **HTTPS**：HTTP的加密安全版本，通过 **SSL/TLS 协议**实现数据加密和身份认证

2. 区别：

1. 加密与安全性：

- HTTP：所有数据均**明文传输**，无加密
- HTTPS：使用 SSL/TLS 协议对数据**加密**，确保数据机密性和完整性

2. 协议与端口：

- HTTP：使用TCP协议，默认端口 80
- HTTPS：
 - 在 HTTP 与 TCP 之间加入 SSL/TLS 层，默认端口 443
 - **TLS 握手**：建立连接时需进行 TLS 握手（1-RTT 或 0-RTT），增加约 1-2 个网络往返延迟

3. 证书与身份验证：

- HTTP：无证书要求，客户端无法验证服务器身份
- HTTPS：
 - **证书**：服务器需部署 CA 签发的证书，证明其域名所有权和合法性
 - **证书链验证**：客户端（浏览器）会验证证书链是否由可信 CA 签发，并检查证书是否过期或被吊销

4. 协议版本：

- HTTP/1.1和HTTP/2：协议本身支持明文传输（HTTP），但主流浏览器强制要求HTTP/2 必须运行在 HTTPS 上
- HTTP/3：必须使用HTTPS

5. 浏览器行为与SEO：

- HTTP：浏览器标记为**不安全**
- HTTPS：显示**安全**标识，**SEO优化**

HTTPS工作原理和加密步骤

1. HTTPS工作原理：

- HTTPS通过在HTTP协议和TCP协议之间加入**SSL/TLS协议层**实现安全传输。确保数据在客户端和服务端之间的传输过程中得到**加密、身份验证和完整性保护**，使数据在传输过程中无法被窃听和篡改。
- 默认端口：443

2. 数据加密步骤：

1. **建立TCP连接**：客户端与服务端首先通过三次握手建立 TCP 连接，这是可靠传输的基础
2. **SSL/TLS握手**：协商加密算法和密钥
 1. **Client Hello（客户端问候）**：客户端发送 Client Hello 消息，包含客户端支持的**SSL/TLS版本、加密算法套件列表、随机数**等信息
 2. **Server Hello（服务器问候）**：服务器从客户端提供的加密算法套件列表选择一个，并返回服务器的**SSL/TLS版本、选择的加密算法套件、随机数**等信息
 3. **Certificate（证书）**：服务器将自己的数字证书发送给客户端。证书包含了服务器的公钥、域名、有效期等信息
 4. **Certificate Verification（证书验证）**：客户端验证服务器的证书是否有效，包括：证书是否有信任的CA颁发；证书是否过期；证书上的域名与服务器域名是否一致
 5. **Key Exchange（密钥交换）**：客户端生成一个**预主密钥**，使用服务器证书中的**公钥**对其进行加密，然后发送给服务器。服务器收到预主密钥后，使用自己的**私钥**解密，获得明文的预主密钥
 6. **Session Key Generation（会话密钥生成）**：客户端和服务端分别使用客户端随机数、服务器随机数和预主密钥，通过相同的算法生成会话密钥。会话密钥用于后续加密数据的对称加密算法
 7. **Change Cipher Spec（更改密码规范）**：客户端和服务端分别发送 Change Cipher Spec 消息，通知对方后续使用**加密通信**
 8. **Finished（完成）**：客户端和服务端分别发送 Finished 消息，**验证握手过程是否成功**
3. **加密通信**：SSL/TLS 握手完成后，客户端和服务端使用会话密钥对后续传输的数据进行加密和解密，保证数据传输的安全

强缓存和协商缓存

强缓存：

1. 定义：

- 浏览器在**本地缓存未过期**的情况下，**直接使用本地缓存资源，不向服务器发送请求**
- 其有效性由服务器响应头中的 **Cache-Control** 或 **Expires** 字段决定
- 若缓存未过期，浏览器直接从本地加载资源，其响应的状态码为 200 OK (from disk/memory cache)

2. 特点：

- **无网络请求**：资源直接从本地缓存加载，无HTTP请求到服务器
- **性能优先**：适用于静态资源（如CSS、JS、图片），显著减少加载时间
- **过期前不可更新**：若资源在缓存有效期内被修改用户无法获取最新版本，过期后进入协商缓存阶段

3. 分类：

- **Cache-Control (HTTP/1.1)**：
 - 通过指令设置**相对过期时间**，优先级高于 **Expires**
 - 指令：max-age、no-cache、no-store、public、private 等
- **Expires (HTTP/1.0)**：
 - 通过绝对时间（GMT格式）指定缓存过期时间
 - Expires: Haha, 31 Feb 2025 00:00:00 GMT

协商缓存：

1. 定义：

- 浏览器**向服务器发送验证请求**，通过比对资源的修改时间（**Last-Modified**）或内容标识（**ETag**）判断缓存是否有效
- 若资源未更新，服务器返回带有 304 Not Modified 响应状态码的HTTP响应，浏览器使用本地缓存
- 若已更新，则返回带有 200 OK 响应状态码 和 **新资源** 的HTTP响应

2. 特点：

- **需网络请求**：每次请求需与服务器通信
- **适用动态资源**：适用于频繁更新的资源（如API响应、实时数据）

3. 分类：

- **Last-Modified / If-Modified-Since**：
 - 首次请求：服务器返回 Last-Modified: Thu, 27 Feb 2025 18:06:00 GMT
 - 再次请求：浏览器携带 If-Modified-Since: Thu, 27 Feb 2025 18:06:00 GMT
 - 服务器收到请求后会对 请求资源 的**最近修改时间** 和 **请求中的 Last-Modified** 首部行进行比较，若服务器的资源发生了更新，则向浏览器返回更新后的资源（200）；若服务器的资源没有更新，就返回 304 状态码而不返回请求的资源
- **ETag / If-None-Match**：
 - 首次请求：服务器返回 ETag: "niubiakamanotes"
 - 再次请求：浏览器携带 If-None-Match: "niubiakamanotes"

- 服务器对比当前资源 ETag，返回 304 或 200

Cookie和Session区别

概念：

1. Cookie：

- **定义：**是服务器发送到浏览器并**存储在本地**的一小段数据（键值对），用于跟踪用户状态。**Cookie** 是浏览器端的存储和传递**载体**（存Session ID或简单数据）
- **作用：**解决 **HTTP 无状态问题**，存储用户偏好或会话标识
- **工作原理：**服务器通过 **Set-Cookie 响应头**下发数据，浏览器后续请求自动通过 **Cookie 请求头**回传

2. Session：

- **定义：**Session 是服务器端维护的会话状态，通过唯一 ID 关联用户请求
- **作用：**存储敏感或临时数据，依赖 Session ID 与浏览器交互
- **工作原理：**服务器生成 Session ID 并通过 Cookie下发，浏览器携带该 ID 以找回对应 Session 数据

区别：

1. 存储位置：

- **Cookie：**数据存储在**浏览器端**，用户可见可修改
- **Session：**数据存储在**服务器端**，仅通过 Session ID 关联浏览器，安全性更高

2. 安全性：

- **Cookie：**安全性低，适用于存储非敏感数据
- **Session：**安全性较高，敏感数据（如用户ID）始终在服务端，仅传递 Session ID

3. 数据类型：

- **Cookie：**值必须是**字符串**，存储对象需手动序列化
- **Session：**值可以是**任意对象**

4. 生命周期：

- **Cookie：**默认**会话级**，浏览器关闭失效，可通过 Expires / Max-Age 设为持久化
- **Session：**默认依赖 Session ID 的 Cookie 生命周期（通常会话级），服务端可主动设置超时

5. 性能：

- **Cookie：**每次请求自动携带，可能增加带宽消耗
- **Session：**占用服务器内存或数据库资源，高并发时需优化存储（如 Redis 集群）

DNS查询过程

1. 查询过程：

- 客户端首先检查**本地缓存**
- 若本地缓存没有命中，客户端会向**本地DNS服务器**发起请求

- 若本地DNS服务器缓存未命中，则依次查询**根域名服务器**、**顶级域名服务器**（如.com）、**权威域名服务器**，最终获取目标域名的IP地址

2. 核心机制：

- **递归查询**：客户端到本地DNS（本地DNS负责完成全部查询）
- **迭代查询**：本地DNS逐级向根、顶级、权威服务器查询（每次返回下一级地址）
- **缓存优化**：各级DNS服务器缓存查询结果，减少全球查询次数。TTL（Time to Live）控制缓存有效期（如3600秒）

从输入URL到页面展示发生了什么

1. URL解析：

浏览器从URL中解析出**主机名**（如 `www.baidu.com`）

2. DNS查询

3. 建立TCP连接

如果是**HTTPS**协议，还需要进行**SSL/TLS**握手

4. 发送HTTP请求

5. 服务器处理并返回响应

6. 浏览器渲染页面

- **解析HTML**：浏览器解析HTML文档，构建DOM树
- **解析CSS**：浏览器解析CSS文件，构建CSSOM树
- **解析执行JavaScript**：浏览器执行JavaScript代码，可能会修改DOM树和CSSOM树
- **构建渲染树**：浏览器将DOM树和CSSOM树合并为渲染树
- **布局**：计算每个元素在屏幕上的精确位置和大小
- **绘制**：把像素填充到屏幕上
- **合成**：多个图层合并，GPU 加速显示